

Graph mining

Ting Kai Ming
Nanjing University

Reading

- Required reading: Aggarwal Chapter 17
- Supplementary reading:
 - Kriege, N.M., Johansson, F.D. & Morris, C. A survey on graph kernels. *Applied Network Science* **5**, 6 (2020). <https://doi.org/10.1007/s41109-019-0195-3> (sections related to Weisfeiler–Lehman (WL) Kernels only)

Acknowledgement:

Unless stated otherwise, the contents and images on the following slides are sourced from C. C. Aggarwal, *Data Mining: The Textbook*. Springer, 2016.

Types of Graph

1. A database of *many small graphs*: chemical and biological data
2. A *single large graph*: the Web and social networks

In a small graph

- Each node is associated with a label that may or may not be unique to the node, depending on the application-specific scenario.
- The repetition of labels within a single graph leads to numerous challenges in graph matching and distance computation.

Some applications

- Chemical and biological data can be expressed as graphs in which each node corresponds to an atom and a bond between a pair of atoms is represented by an edge.
- XML data can be expressed as attributed graphs. The relationships between different attributes of a structured record can be expressed as edges.
- Virtually any data type can be expressed as an entity-relationship graph. This provides a different way of mining conventional database records when they are expressed in the form of entity-relationship graphs.

Graph: Flexibility vs computational complexity

- Graphs lack the “flat” structure of multidimensional or even contextual (e.g., time series) data. Able to model arbitrary relationships between objects.
- The repetition of labels among nodes leads to problems of *isomorphism* in computing similarity between graphs. This problem is NP-hard. This leads to computational challenges in similarity computation and graph matching---fundamental subproblems in graph mining applications.

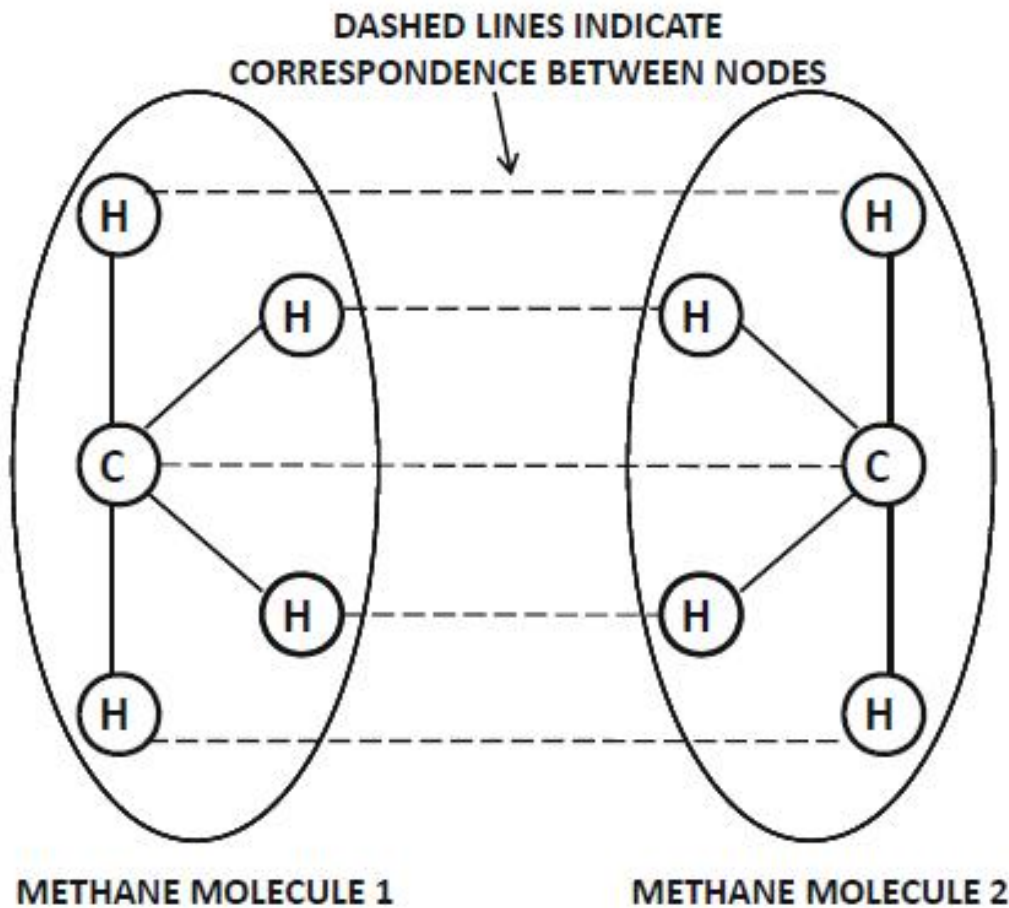
Graph matching between two graphs

Definition 17.2.1 (Graph Matching and Isomorphism) *Two graphs $G_1 = (N_1, A_1)$ and $G_2 = (N_2, A_2)$ are isomorphic if and only if a one-to-one correspondence can be found between the nodes of N_1 and N_2 satisfying the following properties:*

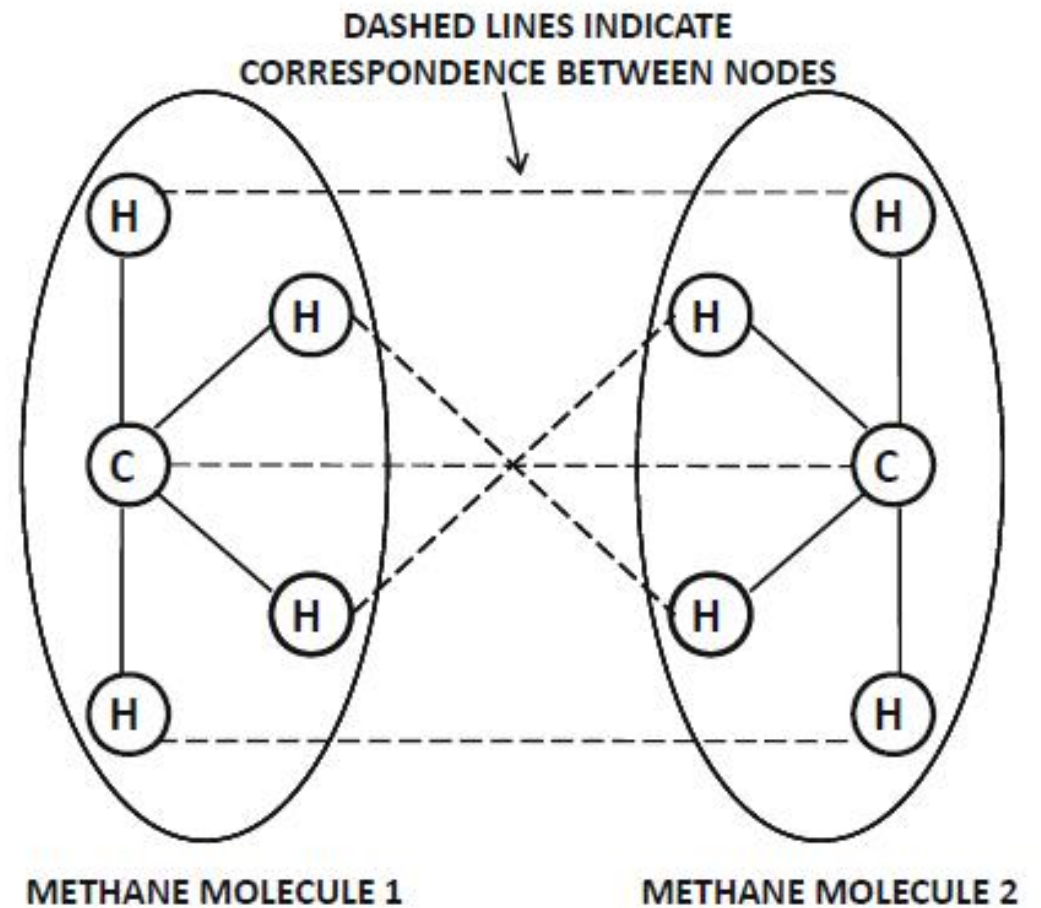
1. *For each pair of corresponding nodes $i \in N_1$ and $j \in N_2$, their labels are the same.*

$$l(i) = l(j)$$

2. *Let $[i_1, i_2]$ be a node-pair in G_1 and $[j_1, j_2]$ be the corresponding node-pair in G_2 . Then the edge (i_1, i_2) exists in G_1 if and only if the edge (j_1, j_2) exists in G_2 .*



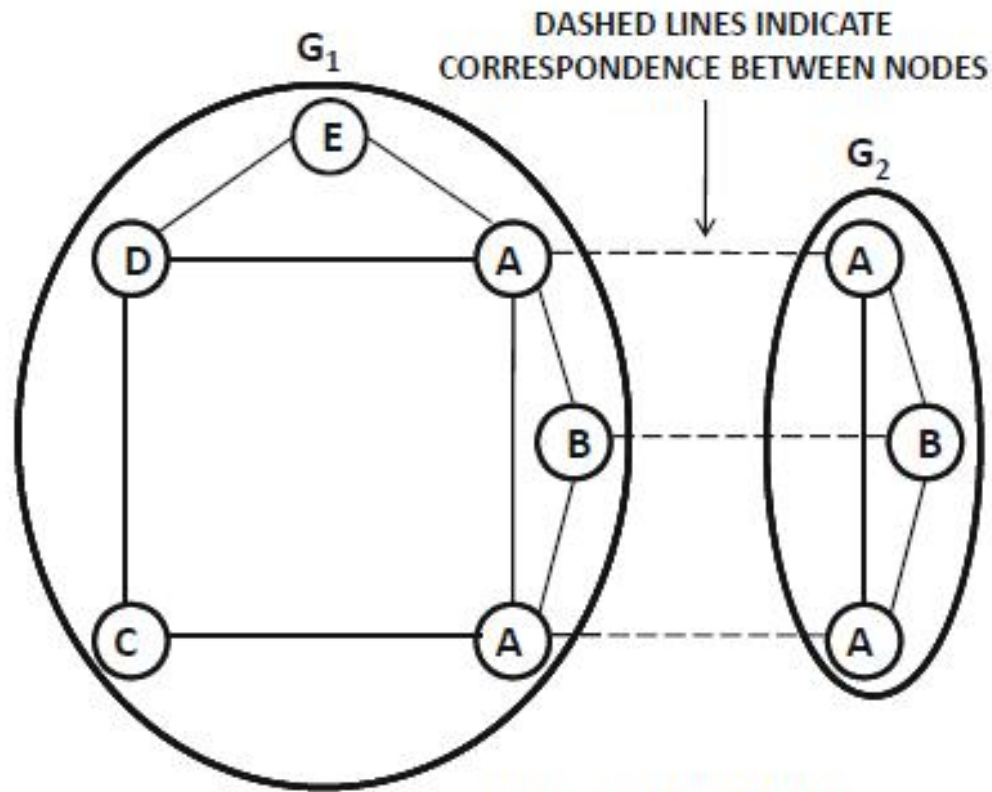
(a)



(b)

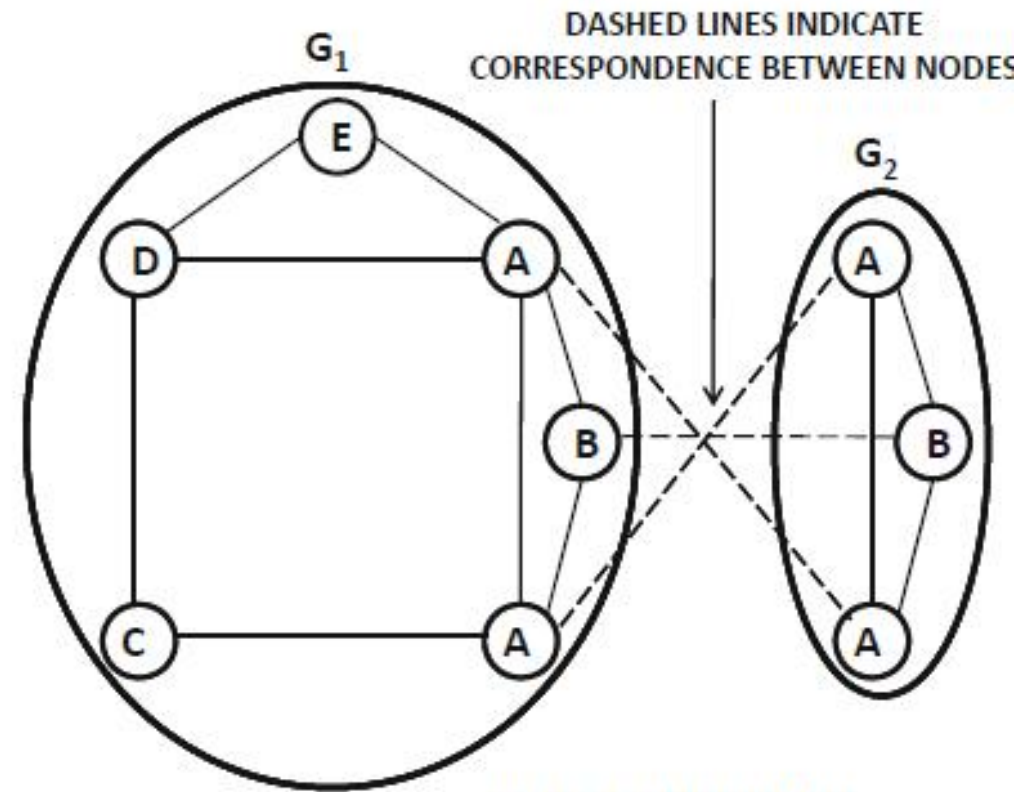
Number of possible matched hydrogen atoms between two graphs: $4!=24$
 The problem of matching a pair of graphs is computationally very expensive.

Subgraph Isomorphism (exact matching)



G_2 IS A SUBGRAPH ISOMORPHISM OF G_1

(a)



G_2 IS A SUBGRAPH ISOMORPHISM OF G_1

(b)

Subgraph Isomorphism algorithms

- Subgraph isomorphism algorithms can be designed to determine either
- *all* the subgraph isomorphisms between a query graph and a data graph (Ullman's algorithm has exponential complexity and is sensitive to the query graph size; pruning is required to reduce the search space), or
 - *whether or not at least one* isomorphism exists (a fast algorithm).

The basic template of Ullman's algorithm

Algorithm *SubgraphMatch*(Query Graph: G_q , Data Graph: G ,
Current Partially Matched Node Pairs: $\mathcal{M}$)

begin

if ($|\mathcal{M}| = |N_q|$) **then return** successful match $\mathcal{M}$;

(Case when $|\mathcal{M}| < |N_q|$ **)**

$\mathcal{C}$ = Set of all label matching node pairs from (G_q, G) not in $\mathcal{M}$;

 Prune $\mathcal{C}$ using heuristic methods; **(Optional efficiency optimization)**

for each pair $(i_q, i) \in \mathcal{C}$ **do**

if $\mathcal{M} \cup \{(i_q, i)\}$ is a valid partial matching

then *SubgraphMatch*($G_q, G, \mathcal{M} \cup \{(i_q, i)\}$);

endfor

end

Maximum common isomorphism between a pair of graphs

Definition 17.2.4 (Maximum Common Subgraph) *A MCG between a pair of graphs $G_1 = (N_1, A_1)$ and $G_2 = (N_2, A_2)$ is a graph $G_0 = (N_0, A_0)$ that is a subgraph isomorphism of both G_1 and G_2 , and for which the size of the node set N_0 is as large as possible.*

- The Ullman algorithm can be extended to the MCG problem.
- The main differences between these methods are in terms of the pruning criteria and the fact that the maximum common subgraph is continuously tracked over the course of the algorithm as the search space of subgraphs is explored.

Maximum common subgraph (MCG) algorithm

Algorithm MCG (**Graphs:** G_1, G_2 , **Current Partially Matched Pairs:** $\mathcal{M}$,
Current Best Match: $\mathcal{M}_{best}$)

begin

$\mathcal{C}$ = Set of all label matching node pairs from (G_1, G_2) not in $\mathcal{M}$;
Prune $\mathcal{C}$ using heuristic methods; (**Optional efficiency optimization**)

for each pair $(i_1, i_2) \in \mathcal{C}$ **do**

if $\mathcal{M} \cup \{(i_1, i_2)\}$ is a valid matching

then $\mathcal{M}_{best} = MCG(G_1, G_2, \mathcal{M} \cup \{(i_1, i_2)\}, \mathcal{M}_{best})$;

endfor

if $(|\mathcal{M}| > |\mathcal{M}_{best}|)$ **then** **return**($\mathcal{M}$) **else** **return**($\mathcal{M}_{best}$);

end

Graph Matching Methods for Distance Computation

- **MCG-based Distances**

Max-normalized distance: $MDist(G_1, G_2) = 1 - \frac{|MCS(G_1, G_2)|}{\max\{|G_1|, |G_2|\}}$

- These distance measures can be computed effectively only for small graphs. For larger graphs, it becomes computationally too expensive to evaluate these measures because of the need to determine the maximum common subgraph between the two graphs.

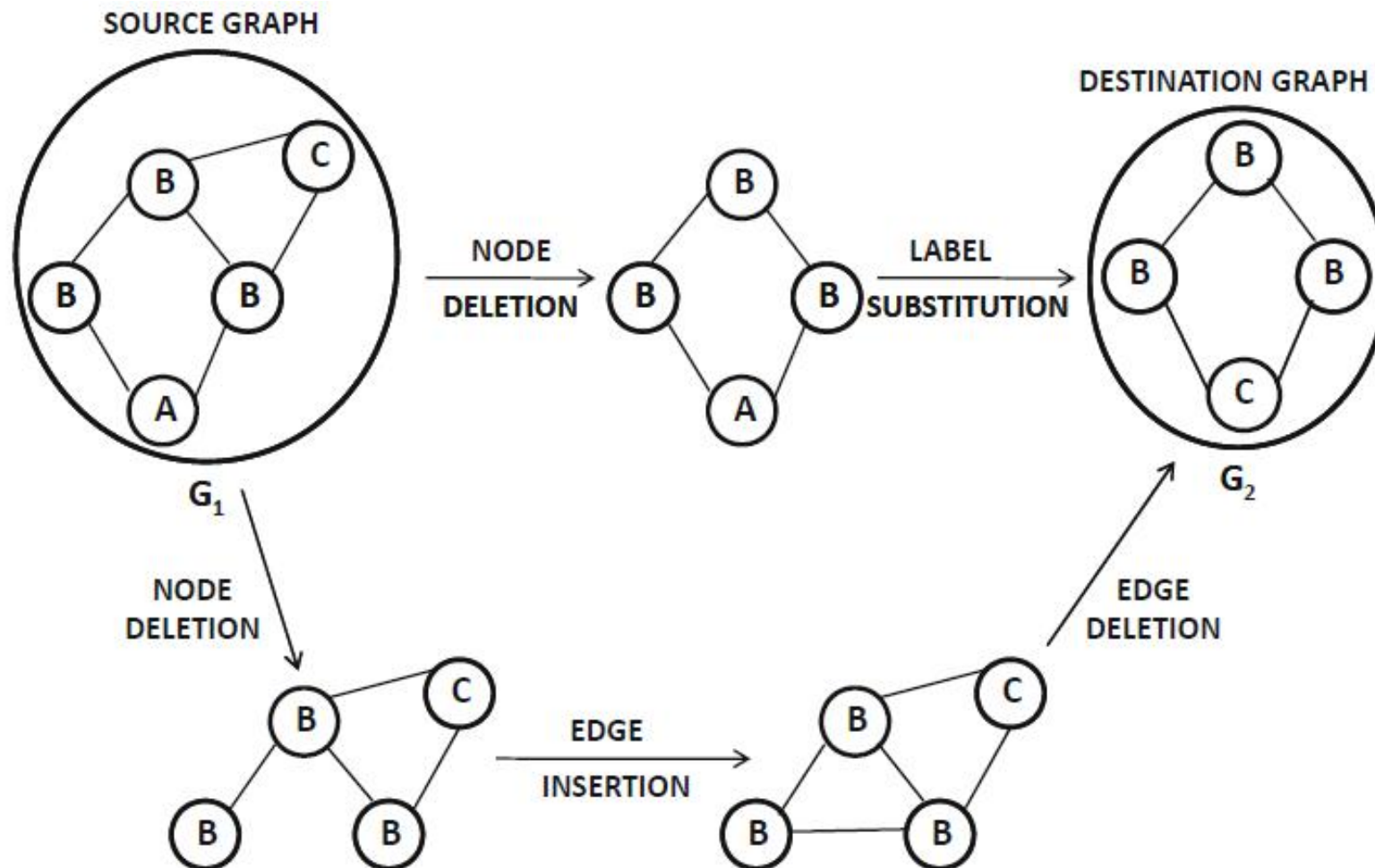
- **Graph Edit Distance**

- The graph edit distance is analogous to the string edit distance

Graph Edit Distance

- The edit distances can be applied to the nodes, the edges, or the labels.
- The admissible operations include (a) the insertion of nodes, (b) the deletion of nodes, (c) the label-substitution of nodes, (d) the insertion of edges, and (e) the deletion of edges.
- Each edit operation has an edit cost associated with it that is defined in an application-specific manner.
- The problem of learning edit costs is a challenging issue in its own right.
- The edit distance $Edit(G_1, G_2)$ between two graphs is equal to the minimum cost of transforming graph G_1 to G_2 with a series of edit operations.
- The edit distance is closely related to the problem of determining MCGs. In fact, for some special choices of costs, the edit distance can be shown to be equivalent to distance measures based on the maximum common subgraph.

Example of two possible edits paths



Graph edit distance algorithm

Algorithm *EditDistance*(Graphs: G_1, G_2 , Current Partial Edit Sequence: $\mathcal{E}$,
Best Known Edit Sequence: $\mathcal{E}_{best}$)

begin

if (G_1 is subgraph isomorphism of G_2) **then begin**

 Add insertion edits to $\mathcal{E}$ that convert G_1 to G_2 ;

return($\mathcal{E}$);

end;

$\mathcal{C}$ = Set of all possible edits to G_1 excluding node-insertions;

 Prune $\mathcal{C}$ using heuristic methods; (**Optional efficiency optimization**)

for each edit operation $e \in \mathcal{C}$ **do**

begin

 Apply e to G_1 to create G'_1 ;

 Append e to $\mathcal{E}$ to create $\mathcal{E}'$;

$\mathcal{E}_{current} = \text{EditDistance}(G'_1, G_2, \mathcal{E}', \mathcal{E}_{best})$;

if ($\text{Cost}(\mathcal{E}_{current}) < \text{Cost}(\mathcal{E}_{best})$) **then** $\mathcal{E}_{best} = \mathcal{E}_{current}$;

endfor

return($\mathcal{E}_{best}$);

end

For a graph with n nodes, the total number of node-based candidate operations is $\mathcal{O}(n)$, and the number of edge-based candidate operations is $\mathcal{O}(n^2)$.

Time complexity

- The procedure is guaranteed to terminate because repetitions in node label-substitutions and edge deletions are avoided in $\mathcal{E}$ by the pruning steps.
- Furthermore, the number of nodes in the edited graph is monotonically non-increasing as more edits are appended to $\mathcal{E}$. This is because $\mathcal{E}$ does not contain node insertions except at the end of the recursion.
- For a graph with n nodes, there are at most $\binom{n}{2}$ non-repeating edge additions and deletions and $O(n)$ node deletions and label-substitutions that can be performed. Therefore, the recursion has a finite depth of $O(n^2)$ that is also equal to the maximum length of $\mathcal{E}$.
- This approach has exponential complexity in the worst case. Edit distances are generally expensive to compute, unless the underlying graphs are small.

Transformation-Based Distance Computation

Methods used to transform the graphs into a space in which distance computations are more efficient. Interestingly, some of these methods are also *qualitatively* more effective because of their ability to focus on the relevant portions of the graphs.

- 1. Frequent Substructure-Based Transformation and Distance Computation**
- 2. Topological Descriptors**
- 3. Kernel-Based Transformations and Computation**

Frequent Substructure-Based Transformation

- The intuition underlying this approach is that frequent graph patterns encode key properties of the graph: in terms of its family of frequent substructures.
- A text-like vector-space representation is created from each graph.

The steps:

1. Apply a frequent subgraph mining method to discover frequent subgraph patterns in the underlying graphs. This results in a “lexicon” in terms of which the graphs are represented.
2. Select a subset of subgraphs from the subgraphs found in the first step to reduce the overlap among the frequent subgraph patterns. Create a new feature f_i for each frequent subgraph S_i that is finally selected. Let d be the total number of frequent subgraphs (features). This is the lexicon size in terms of which a text-like representation will be constructed.
3. For each graph G_i , create a vector-space representation in terms of the features $f_1 \dots f_d$. The frequency of each feature is the number of occurrences of the corresponding subgraph in the graph G_i .

Advantage and disadvantage

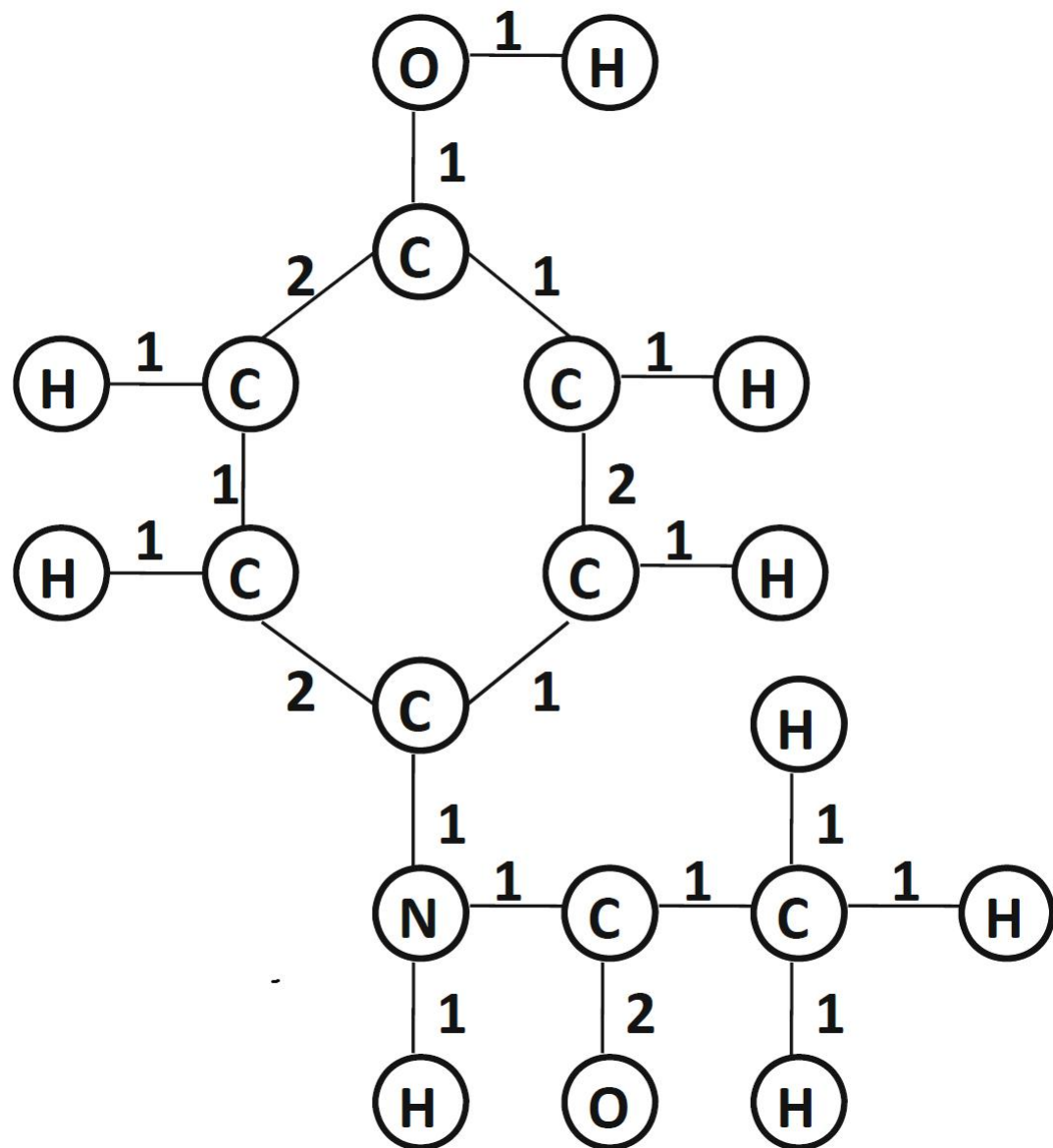
- Any data mining algorithm from the text domain can be applied to graphs using this approach.
- The main disadvantage of this approach is that subgraph isomorphism is an intermediate step in the frequent substructure discovery.
- The approach has exponential complexity in the worst case.

Topological Descriptors

- Topological descriptors convert structural graphs to multidimensional data by using quantitative measures of important structural characteristics as dimensions.
- The drawback of the approach is that the structural information is lost.
- Most topological descriptors are graph specific, whereas a few are node-specific.
Some common examples:
 1. *Morgan index*: This is a node-specific index that is equal to the k th order degree of a node.
 2. *Wiener index*: The Wiener index is equal to the sum of the pairwise shortest path distances between all pairs of nodes.
 3. *Hosoya index*: The Hosoya index or Z-index is equal to the number of valid pairwise node matchings within the same graph.
 4. *Estrada index*: This index is particularly useful in chemical applications for measuring the degree of protein folding.
 5. *Circuit rank*: The circuit rank $C(G)$ is equal to the minimum number of edges that need to be removed from a graph in order to remove all cycles.
 6. *Randic index*: The Randic index is equal to the pairwise sum of bond contributions.

Discussion #1

1. *Consider two graphs, which are cliques containing an even number $2 \cdot n$ nodes. Let exactly half the nodes in each graph belong to labels A and B. What are the total number of isomorphic matchings between the two graphs?*
2. *Consider two graphs, containing $2 \cdot n$ nodes and n distinct labels, each occurring twice. What is the maximum number of isomorphic matchings between the two graphs?*
3. *Compute the Morgan Indices of order 1 and 2, for each node of the Acteominophen graph. How does the Morgan Index vary with the label (corresponding to chemical element)?*



Kernel-Based Transformations

- The kernel similarity $K(G_i, G_j)$ between a pair of graphs G_i and G_j is the dot product of the two graphs after hypothetically transforming them to a new space, defined by the function $\Phi(\cdot)$.

$$K(G_i, G_j) = \Phi(G_i) \cdot \Phi(G_j)$$

- In practice, (for some methods) the value of $\Phi(\cdot)$ is not defined directly. Rather, it is defined indirectly in terms of the kernel similarity function $K(\cdot, \cdot)$.

Some common kernels:

1. **Random Walk Kernels**
2. **Shortest-Path Kernels**
3. **Weisfeiler–Lehman Kernels**

Random Walk Kernels

- The idea is to compare the label sequences induced by random walks in the two graphs. Intuitively, two graphs are similar if many sequences of labels created by random walks between pairs of nodes are similar as well.
- The main computational challenge is that there are an exponential number of possible random walks between pairs of nodes, and the length of a walk may be any value in the range $(1, \infty)$.
- $K(G_1, G_2)$ is defined as the sum of the probabilities of all the primitive sequence kernels over all possible walks:

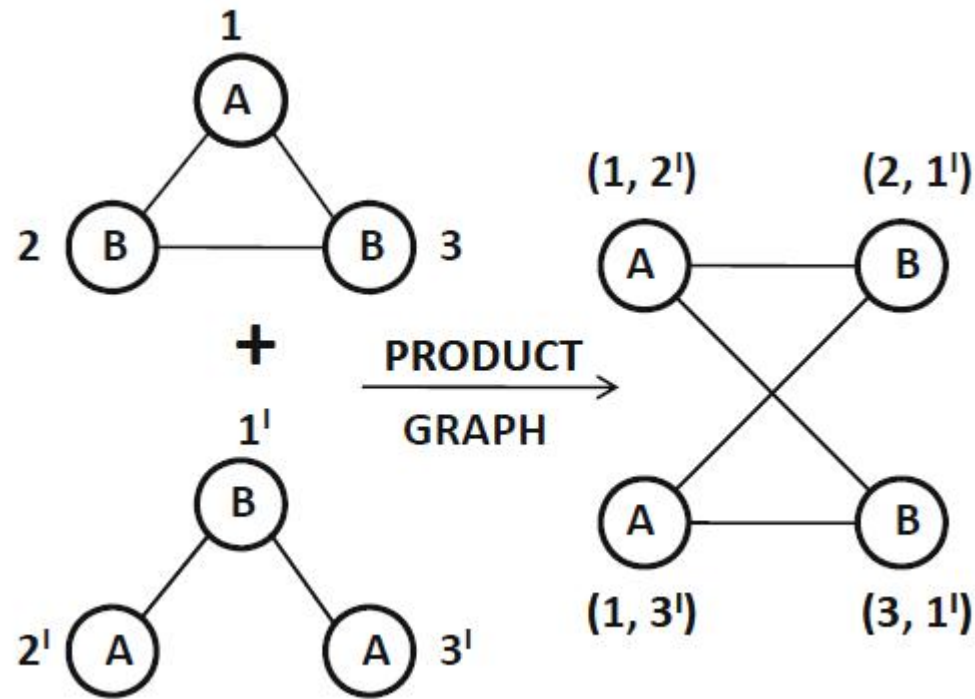
$$K(G_1, G_2) = \sum_{s_1, s_2} p(s_1/G_1) \cdot p(s_2/G_2) \cdot k(s_1, s_2)$$

$p(s_i/G_i)$ is the probability of the random walk sequence s_i in the graph G_i .

$$k(s_1, s_2) = \mathbb{I}(s_1 = s_2)$$

Product Graph

- The random walk kernel is computed using the notion of a *product graph* G_x between G_1 and G_2 .



Examples of Random walk kernels

Each walk in the product graph corresponds to a pair of label-matching sequence of vertices in the two graphs G_1 and G_2 . Then, if A is the adjacency matrix of the product graph, then the entries of A^k provide the number of walks of length k between the different pairs of vertices. Therefore, the total weighted number of walks may be computed as follows:

$$K(G_1, G_2) = \sum_{i,j} \sum_{k=1}^{\infty} \lambda^k [A^k]_{ij} = \bar{e}^T (I - \lambda A)^{-1} \bar{e}$$

$$K(G_1, G_2) = \sum_{i,j} \sum_{k=1}^{\infty} \frac{\lambda^k}{k!} [A^k]_{ij} = \bar{e}^T \exp(\lambda A) \bar{e}$$

$\bar{e}$ is an $|G_x|$ -dimensional column vector of 1s, and $\lambda \in (0, 1)$ is a discount factor.

Shortest-Path Kernels

- In the shortest-path kernel, a primitive kernel $k_s(i_1, i_2, j_1, j_2)$ is defined on node-pairs $[i_1, j_1] \in G_1$ and $[i_2, j_2] \in G_2$. There are several ways of defining the kernel function $k_s(i_1, i_2, j_1, j_2)$. A simple way of defining the kernel value is to set it to 1 when the distance $d(i_1, j_1) = d(i_2, j_2)$, and 0, otherwise.
- The overall kernel similarity is equal to the sum of all primitive kernels over different quadruplets of nodes:

$$K(G_1, G_2) = \sum_{i_1, i_2, j_1, j_2} k_s(i_1, i_2, j_1, j_2).$$

- The shortest-path kernel may be computed by applying the all-pairs shortest-path algorithm on each of the graphs. It can be shown that the complexity of the kernel computation is $O(n^4)$. Although this is still quite expensive, it may be practical for small graphs, such as chemical compounds.

Weisfeiler–Lehman (WL) Kernels

- WL kernels use the neighborhood aggregation approach
- Paradigm in the design of graph kernel: representation and comparison of *local* structure.
- The idea of the *Weisfeiler-Lehman subtree kernel* is to compute the color refinement algorithm (Weisfeiler-Lehman (WL) relabeling) for $h \geq 0$ iterations, and after each iteration i compute a feature vector $\varphi^i(G) \in \mathbb{R}^{|\Sigma^i|}$ for each graph G , where $\Sigma^i \subseteq \Sigma$ (label dictionary). The overall feature vector $\varphi_{\text{WL}}(G)$ is defined as the concatenation of the feature vectors of all h iterations

$$\varphi_{\text{WL}}(G) = [\varphi^0(G), \varphi^1(G), \dots, \varphi^h(G)]^T$$

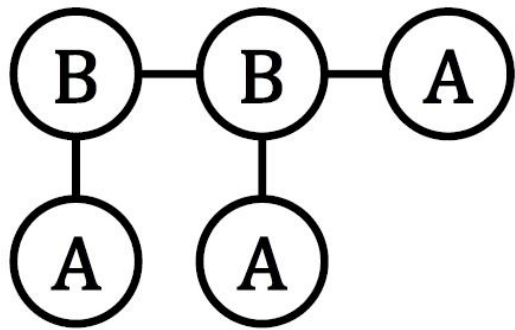
$$K_{\text{WL}}(G, H) = \langle \varphi_{\text{WL}}(G), \varphi_{\text{WL}}(H) \rangle$$

This is for labeled (non-attributed) graphs.

Example of Weisfeiler-Lehman (WL) relabeling

Original labels

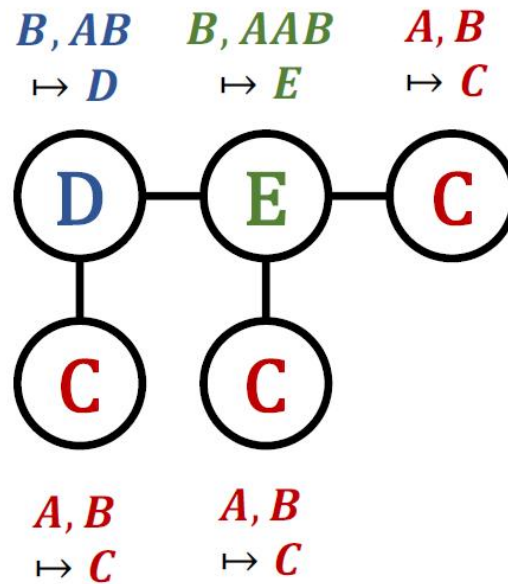
$i = 0$



$\Sigma = \{A, B\}$

Relabeled

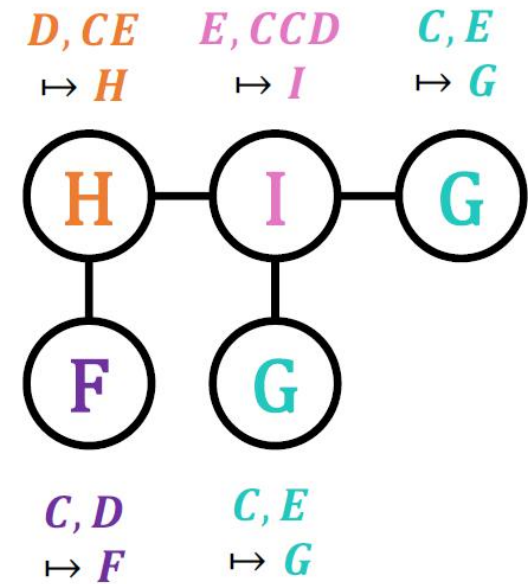
$i = 1$



$\Sigma = \{A, B, C, D, E\}$

Relabeled

$i = 2$



$\Sigma = \{A, B, C, D, E, F, G, H, I\}$

...

Variants of WL kernels

After each WL iteration:

- WL subtree kernel: counts the labels of vertices.
- WL edge kernel: counts the colors of the two endpoints for all edges.
- WL shortest-path kernel: the sum of shortest-path kernels applied to the graphs with refined labels $l^{(i)}$ for $i \in \{0, \dots, h\}$.

'... one of the great contributions of the Weisfeiler-Lehman and related kernels (e.g. Propagation kernels) is that they capture such dependencies (between labels and structure) in transformed graphs that are beneficial to all kernels that support vertex labels. It has therefore become standard practice to perform a WL-like transform on labeled graphs before application of other kernels.'

Kriege, Johansson & Morris. (2020)

Weisfeiler–Lehman (WL) transformation for attributed graphs

- The key idea is to create an explicit propagation scheme that leverages and updates the current node features by averaging over the neighborhoods.
- Let every node v in a graph has continuous attributes $a(v)$. Let $a^0(v) = a(v)$, the attributes for v defined recursively at iteration $h+1$ as follows:

$$a^{h+1}(v) = \frac{1}{2} \left(a^h(v) + \frac{1}{\deg(v)} \sum_{u \in \mathcal{N}(v)} w((v, u)) \cdot a^h(u) \right)$$

where $N(v)$ is the neighborhood of v ; w is the edge weight between two nodes.

Let graph $G = (V; E)$ & $a(v) \in R^m$, let h be the number of WL iterations.

The WL features for node $v \in V$ is defined as $\Phi(v) \in R^{m \times (h+1)}$:

$$\Phi(v) = [a^0(v), \dots, a^h(v)]^T$$

Isolation Graph Kernel ^[*]

- Layer #1: Dataset-wide node embedding using IK
- Layer #2: Graph-specific node embedding using a Weisfeiler–Lehman scheme
- Layer #3: Mean embedding of a graph

Algorithm 1: Compute IGK mean map

Input : Graph dataset $\{G_i = (V_i, E_i), i = 1, \dots, n\}$,
IK parameter ψ , WL parameter h .

Output: IGK mapped dataset $\{\hat{\Phi}(G_i), i = 1, \dots, n\}$.

```
1 D =  $\cup_{v \in V_i, i \in [1, n]} \mathbf{v}$  /* Set of node vectors */
2 Produce  $\varphi(\cdot | \mathbf{D})$  from D /* Embedding #1 */
3 for ( $i = 1$ ;  $i > n$ ;  $i++$ ) do
4   for (each  $v \in V_i$ ) do
5     /* Embedding layer #2 */
6     Map  $\varphi^{j-1}(\mathbf{v})$  into  $\varphi^j(\mathbf{v})$  for  $j = 1, \dots, h$ ;
7      $\Phi(\mathbf{v}) = [\varphi^0(\mathbf{v}), \dots, \varphi^h(\mathbf{v})]^\top$ ;
8   end
9   /* Embedding layer #3 */
10   $\hat{\Phi}(G_i) = \frac{1}{|V_i|} \sum_{v \in V_i} \Phi(\mathbf{v})$ ;
11 end
12 return  $\{\hat{\Phi}(G_i), i = 1, \dots, n\}$ 
```

[*] Bi-Cun Xu, Kai Ming Ting, Yuan Jiang (2021). Isolation Graph Kernel.
Proceedings of The Thirty-Fifth AAAI Conference on Artificial Intelligence. 10487-10495.

Graph Clustering

- The graph clustering problem partitions a database of n graphs, denoted by $G_1 \dots G_n$, into groups.

Methods:

- Distance-Based Methods: more effective for smaller graphs in which distances can be computed robustly and efficiently.
- Frequent substructure-based methods are appropriate for larger graphs where distance computations become qualitatively and computationally impractical.

Distance-Based Clustering

- k -medoids and spectral clustering: distance function design is the most fundamental problem that needs to be addressed

These methods do not work very well when the individual graph objects are large because of two reasons:

- It is generally computationally expensive to compute distances between large graph objects. Graph distance functions, such as matching-based methods, have a complexity that increases exponentially with graph object size.
- The *effectiveness* of such methods also drops sharply with increasing graph size. This is because the graphs may be similar only in some portions that repeat frequently. The rare portions of the graphs may be unique to the specific graph at hand. Therefore, a matching-based distance function may not be able to properly compare the key features of the different graphs.

K-Medoids clustering

- A set of k representatives is used, and data objects are assigned to their closest representatives by using an appropriately designed distance function.
- The set of k representatives is progressively optimized by using a hill climbing approach, in which the representatives are iteratively swapped with other data objects in order to improve the clustering objective function value.

Spectral Clustering

- The individual graph objects are used to construct a single large neighborhood graph. The latter graph is a higher level similarity graph, in which each node corresponds to one of the (smaller) graph objects from the original database and the weight of the edge is equal to the similarity between the two objects.
- Each node is connected to its k -nearest neighbors with an undirected edge. Thus, the problem of clustering graph *objects* is transformed to the problem of clustering *nodes* in a single large graph.
- The details of Spectral-Clustering will be examined in Social Network Analysis

Graph Classification

- A set of n graphs $G_1 \dots G_n$ is available, but only a subset of these graphs is labeled. The task is to use the labels on the training graphs to infer the labels of unlabeled graphs.

Methods:

- **Distance-Based Methods**
- **Frequent Substructure-Based Methods**
- **Kernel-based methods**

Distance-Based Classification

- *Nearest neighbor methods*: For each test instance, the k -nearest neighbors are determined. The dominant label from these nearest neighbors is reported as the relevant label.
- *Graph-based methods*: This is a transductive semi-supervised method. A higher level *neighborhood graph* is constructed from the training and test graph objects. The original graph objects correspond to nodes in the neighborhood graph. Each node is connected to its k nearest neighbor objects based on the distance values. This results in a graph containing both labeled and unlabeled nodes. Collective classification algorithms can be used to derive labels of the nodes in the neighborhood graphs. (We will re-visit these methods in Social Network Analysis)

Kernel-based methods

- Graph kernels can be used in SVM and other kernel-based classifiers by simply replacing the existing kernel with a chosen graph kernel to perform graph classification.

Reflection

- What have you learned this week?
(Every student shall attempt to answer this question.)

Summary

- Graph data are a challenging domain for analysis, because of the difficulty in matching two graphs when there are repetitions in the underlying labels. This is referred to as graph isomorphism. Most methods for graph matching require exponential time in the worst case.
- Two general distance measures of two graphs:
 - Maximum Common subgraph (MCG) and edit-distance
 - Transform the graph database into a simpler text-like representation
- Frequent substructure-based methods
 - An important class of graph similarity functions is graph kernels.
 - Many clustering and classification algorithms can be used once a distance or kernel is defined for graphs. Examples are nearest neighbor methods and SVM.
- Directly mine the frequent substructures and use them as representative sets of clusters, or antecedents of discriminative rules.

Discussion #2

1. In general, a kernel-based method is preferred over distance-based method (e.g., in the context of classification). Explain why this is so.
2. Describe how a single large neighborhood graph can be constructed from a dataset of graphs in order to use spectral clustering. (Hint: the first issue you need to resolve is: how to represent a graph.)
3. Describe the difference between labeled (non-attributed) graphs and attributed graphs.
4. Graph kernels can be categorized into two kinds of feature mapping: implicit and explicit. Which category does each of Random-walk kernel, shortest path kernel and WL kernel belong? Explain your answer in relation to $K(G_i, G_j) = \Phi(G_i) \cdot \Phi(G_j)$
5. Consider the three graph kernels and the generic transformational (GT) approach we have studied. Can these graph kernels be used with GT? Explain the reason why.

Discussion #3

1. Why k-medoids are used instead of k-means in the graph domain?
2. Given your reason stated for Question 1, would kernel k-means be a valid method when a graph kernel is used to measure similarity between two graphs?

Discussion #4

1. Given what you have learned about graph kernels, explain how you will use them to perform data mining (i.e., anomaly detection, clustering or clustering) in each of the following problems:

- a) A dataset of small graphs

- b) A large network

Explain the necessary ingredients in order to perform a data mining task, e.g., which graph kernel (or any graph kernel), algorithm to use, etc.

2. From the graph kernels we have studied, identify which one does or does not take the distribution of node vectors in an attributed network into consideration.